

```

from anthropic import Anthropic
from dotenv import load_dotenv
import os
import subprocess
from pathlib import Path

load_dotenv(override=True)
client=Anthropic(base_url=os.getenv("ANTHROPIC_BASE_URL"))
MODEL=os.getenv("MODEL_ID")
# 修复: 定义工作目录 (默认当前目录, 可通过环境变量覆盖)
WORKDIR = Path(os.getenv("AGENT_WORKDIR", os.getcwd())).resolve()
WORKDIR.mkdir(parents=True, exist_ok=True)

SYSTEM=f"you are a coding agent at{os.getcwd()}.Use tools to solve tasks.act.don't explain"

TOOLS = [
    {
        "name": "bash",
        "description": "Run a shell command.",
        "input_schema": {
            "type": "object",
            "properties": {"command": {"type": "string", "description": "Shell command to execute"}},
            "required": ["command"],
        }
    },
    {
        "name": "read_file",
        "description": "Read file contents.",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {"type": "string", "description": "Relative path to file"},
                "limit": {"type": "integer", "description": "Max lines to read, optional"}
            },
            "required": ["path"],
        }
    },
    {
        "name": "write_file",
        "description": "Write content to a file.",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {"type": "string", "description": "Relative path to file"},
                "content": {"type": "string", "description": "File content to write"}
            },
            "required": ["path", "content"],
        }
    },
    {

```

```

        "name": "edit_file",
        "description": "Replace exact text in a file once.",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {"type": "string", "description": "Relative path to file"},
                "old_text": {"type": "string", "description": "Text to replace"},
                "new_text": {"type": "string", "description": "Replacement text"}
            },
            "required": ["path", "old_text", "new_text"],
        },
    },
    {
        "name": "glob",
        "description": "Find files matching a glob pattern; ** matches recursively.",
        "input_schema": {
            "type": "object",
            "properties": {"pattern": {"type": "string", "description": "Glob pattern, e.g.
**/*.py"}},
            "required": ["pattern"],
        },
    },
]

```

```

def safe_path(p: str) -> Path:
    path = (WORKDIR / p).resolve()
    if not path.is_relative_to(WORKDIR):
        raise ValueError(f"Path escapes workspace: {p}")
    return path

def run_bash(command:str)->str:
    dangerous=["rm -rf /","sudo","shutdown","reboot",> /dev/"]
    if any(d in command for d in dangerous):
        return "Error: Dangerous command blocked"
    try:
        r=subprocess.run(command,shell=True,cwd=os.getcwd(),
                           capture_output=True,text=True,errors="replace",
                           timeout=120)
        out=(r.stdout+r.stderr).strip()
        return out[:50000] if out else "no output"
    except subprocess.TimeoutExpired:
        return "Error:Timeout(120s)"
    except (FileNotFoundError,OSError) as e:
        return f"Error:{e}"

def run_read(path: str, limit: int | None = None) -> str:
    try:
        lines = safe_path(path).read_text(encoding="utf-8").splitlines()
        if limit and limit < len(lines):
            lines = lines[:limit] + [f"... ({len(lines) - limit} more lines)"]
        return "\n".join(lines)
    except Exception as e:

```

```

        return f"Error: {e}"

def run_write(path: str, content: str) -> str:
    try:
        file_path = safe_path(path)
        file_path.parent.mkdir(parents=True, exist_ok=True)
        file_path.write_text(content, encoding="utf-8")
        return f"Wrote {len(content)} bytes to {path}"
    except Exception as e:
        return f"Error: {e}"

def run_edit(path: str, old_text: str, new_text: str) -> str:
    try:
        file_path = safe_path(path)
        text = file_path.read_text(encoding="utf-8")
        if old_text not in text:
            return f"Error: text not found in {path}"
        file_path.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
        return f"Edited {path}"
    except Exception as e:
        return f"Error: {e}"

def run_glob(pattern: str) -> str:
    import glob as g
    try:
        matches = sorted({
            match for match in g.glob(pattern, root_dir=WORKDIR, recursive=True)
            if (WORKDIR / match).resolve().is_relative_to(WORKDIR)
        })
        shown = matches[:200]
        if len(matches) > 200:
            shown.append("... (more matches omitted; narrow the pattern)")
        return "\n".join(shown) if shown else "(no matches)"
    except Exception as e:
        return f"Error: {e}"

TOOL_HANDLERS = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
}

def agent_loop(messages: list):
    while True:
        # 1.把帐本发给模型
        response=client.messages.create(
            model=MODEL,system=SYSTEM,messages=messages,
            tools=TOOLS,max_tokens=8000
        )
        # 2.模型的回复记录进账本

```

```

messages.append({"role": "assistant", "content": response.content})
print(f"模型回复的所有内容: {response.content}")

# 3. 这轮有没有调用工具? 没有任务完成退出
tool_calls=[
    block for block in response.content if block.type=="tool_use"
]

if not tool_calls:
    return

results=[]
for block in tool_calls:
    handler = TOOL_HANDLERS.get(block.name)
    output = handler(**block.input) if handler else f"Unknown: {block.name}"
    results.append({
        "type": "tool_result",
        "tool_use_id": block.id,
        "content": output,
    })

# 5. 结果塞回去账本, 回到第一步
messages.append({"role": "user", "content": results})

if __name__=="__main__":
    print("s01 agent loop")
    print("请输入一个问题, 点击enter发送, 点击q会退出。\\n")

    history=[]
    while True:
        try:
            query=input("agent>>")
        except (EOFError, KeyboardInterrupt):
            break
        if query.strip().lower() in ("q", "exit", ""):
            break

        history.append({"role": "user", "content": query})
        agent_loop(history)

        for block in history[-1]["content"]:
            if getattr(block, "type", None)=="text":
                print(block.text)

    print()

```

